

#Experiment 4(a):

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.datasets import make_regression

from sklearn import metrics

from sklearn.linear_model import LinearRegression


#import make_regression dataset with from sklearn

n_samples=100

n_features=1

bias = 0

noise=5

x, y = make_regression(n_samples=n_samples, n_features=n_features, bias=bias, noise=noise)

#Scale feature x to range -5.....5

x = np.interp(x, (x.min(), x.max()), (-5, 5))

# Scale target y to range 15.....-15

y = np.interp(y, (y.min(), y.max()), (15, -15))


# Split the dataset into 80% training and 20% testing sets

test_size=0.2

n_train = int(n_samples*(1-test_size))

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=test_size)


# scatter plot for overall dataset, training dataset and testing dataset

plt.scatter(x, y, edgecolors='k', marker='o', label='Overall Dataset')

plt.legend()

plt.xlabel('Feature x')

plt.ylabel('Target y')

plt.title('DataSet Display')
```

```

plt.show()

plt.scatter(x_train, y_train, edgecolors='g', marker='*', label='Training Dataset')

plt.scatter(x_test, y_test, edgecolors='r', marker='^', label='Testing Dataset')

plt.legend()

plt.xlabel('Feature x')

plt.ylabel('Target y')

plt.title('Linear Regression Display')


# Add bias term for each instance in training dataset
x_train_b = np.c_[np.ones((n_train, 1)), x_train]


# Linear Regression Training using Closed Form Solution
y1 = np.linalg.inv(np.matmul(x_train_b.T, x_train_b))
y2 = np.matmul(x_train_b.T, y_train)
w_closed_form = np.matmul(y1, y2)

print('Linear Regression Co-efficient [w0, w1] using Closed Form Solution = ')

print(w_closed_form)


# Plotting the linear regression line

plt.plot(x_train, w_closed_form[0]+w_closed_form[1]*x_train, 'r-', label='Linear Regression (Closed Form)')

plt.legend()


# RMSE for training data set

rmse_train =
(np.sqrt(metrics.mean_squared_error(y_train, w_closed_form[0]+w_closed_form[1]*x_train)))

print('RMSE for Training Linear Regression with Closed Form Solution =')

print(rmse_train)


# Linear Regression Testing and Performance Evaluation using Closed Form Solution
y_pred = w_closed_form[0]+w_closed_form[1]*x_test
rmse_test = (np.sqrt(metrics.mean_squared_error(y_test, y_pred)))

```

```

print('RMSE for Testing Linear Regression with Closed Form Solution =')
print(rmse_test)

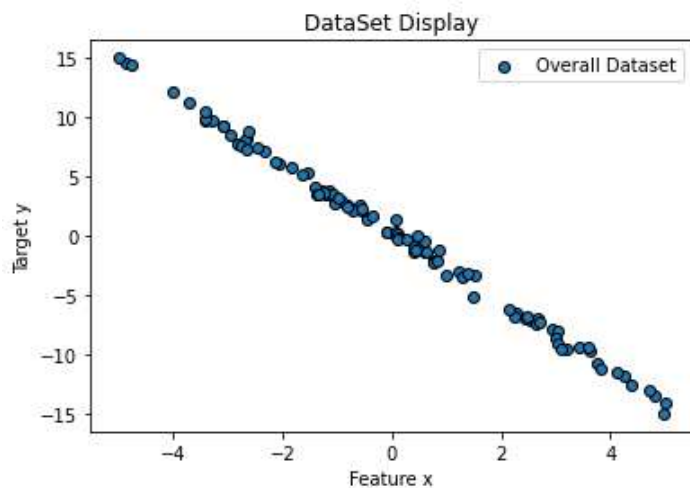
# Verification using Sklearn in-built function for linear regression
reg = LinearRegression().fit(x_train,y_train)
w_sklearn = [reg.intercept_, reg.coef_]
print("Linear Regression Co-efficient [w0, w1] using sklearn in-built function")
print(w_sklearn)

plt.plot(x_train, w_sklearn[0]+w_sklearn[1]*x_train, 'c-', label='Linear Regression (Skelearn in-built function)')

plt.legend()
plt.show()

#Results:

```



```

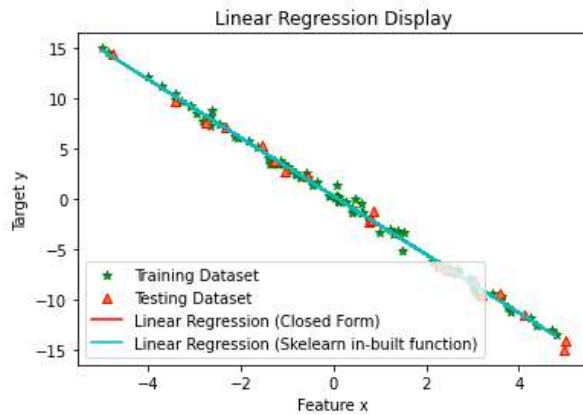
Linear Regression Co-efficient [w0, w1] using Closed Form Solution =
[ 0.29126771 -2.9062105 ]

RMSE for Training Linear Regression with Closed Form Solution =
0.4243328127926739

RMSE for Testing Linear Regression with Closed Form Solution =
0.5071085073457491

Linear Regression Co-efficient [w0, w1] using sklearn in-built function
[0.29126770891684783, array([-2.9062105])]

```



#Experiment 4(b):

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
from sklearn import metrics
from sklearn.linear_model import Ridge

#import make_regression dataset with from sklearn
n_samples=100
n_features=1
bias = 0
noise=5

x, y = make_regression(n_samples=n_samples, n_features=n_features, bias=bias, noise=noise)

#Scale feature x to range -5.....5
x = np.interp(x, (x.min(), x.max()), (-5, 5))

# Scale target y to range 15.....-15
y = np.interp(y, (y.min(), y.max()), (15, -15))

# Split the dataset into 80% training and 20% testing sets
test_size=0.2

n_train = int(n_samples*(1-test_size))

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=test_size)
```

```

# scatter plot for overall dataset
plt.scatter(x, y, edgecolors='k', marker='o', label='Overall Dataset')

plt.legend()

plt.xlabel('Feature x')
plt.ylabel('Target y')
plt.title('DataSet Display')
plt.show()

# Add bias term for each instance in training dataset
x_train_b = np.c_[np.ones((n_train, 1)), x_train]

# Regularization parameter
lambda = [0.1, 0.5, 1.0, 1.5]
for i, alpha in enumerate(lambda):
    print("regularization term = ")
    print(alpha)

# scatter plot for training dataset and testing dataset
plt.scatter(x_train, y_train, edgecolors='g', marker='*', label='Training Dataset')
plt.scatter(x_test, y_test, edgecolors='r', marker='^', label='Testing Dataset')
plt.legend()
plt.xlabel('Feature x')
plt.ylabel('Target y')
plt.title('Ridge Regression Display')

# Ridge Regression Training using Closed Form Solution
y1 = np.linalg.inv(np.matmul(x_train_b.T, x_train_b) + alpha * np.eye(x_train_b.shape[1]))
y2 = np.matmul(x_train_b.T, y_train)
w_closed_form = np.matmul(y1, y2)
print('Ridge Regression Co-efficient [w0, w1] using Closed Form Solution = ')
print(w_closed_form)

```

```

# Plotting the linear regression line

plt.plot(x_train, w_closed_form[0]+w_closed_form[1]*x_train, 'r-', label=f'Ridge Regression
(Closed Form) with lambda={alpha}')

plt.legend()

# RMSE for training data set

rmse_train =
(np.sqrt(metrics.mean_squared_error(y_train,w_closed_form[0]+w_closed_form[1]*x_train)))

print('RMSE for Training Ridge Regression with Closed Form Solution =')

print(rmse_train)

# Ridge Regression Testing and Performance Evaluation using Closed Form Solution

y_pred = w_closed_form[0]+w_closed_form[1]*x_test

rmse_test = (np.sqrt(metrics.mean_squared_error(y_test,y_pred)))

print('RMSE for Testing Ridge Regression with Closed Form Solution =')

print(rmse_test)

# Verification using Sklearn in-built function for linear regression

ridge_model = Ridge(alpha=alpha)

reg = ridge_model.fit(x_train,y_train)

w_sklearn = [reg.intercept_, reg.coef_]

print("Ridge Regression Co-efficient [w0, w1] using sklearn in-built function")

print(w_sklearn)

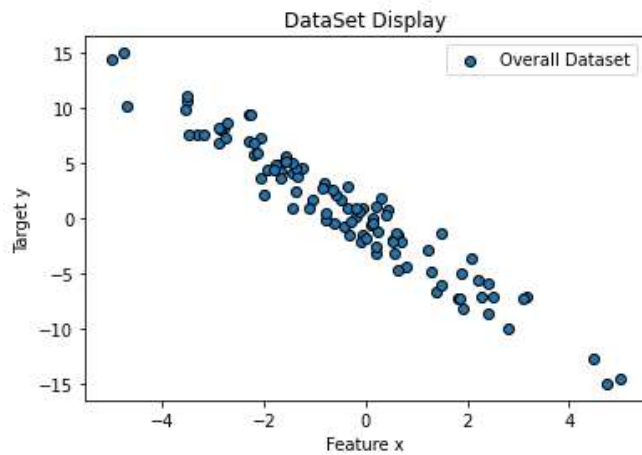
plt.plot(x_train, w_sklearn[0]+w_sklearn[1]*x_train, 'c-', label=f'Ridge Regression (Sklearn in-built
function) with lambda={alpha}')

plt.legend()

plt.show()

#Results:

```



regularization term =

0.1

Ridge Regression Co-efficient $[w_0, w_1]$ using Closed Form Solution =

$[-0.32843203 \ -2.8797737]$

RMSE for Training Ridge Regression with Closed Form Solution =

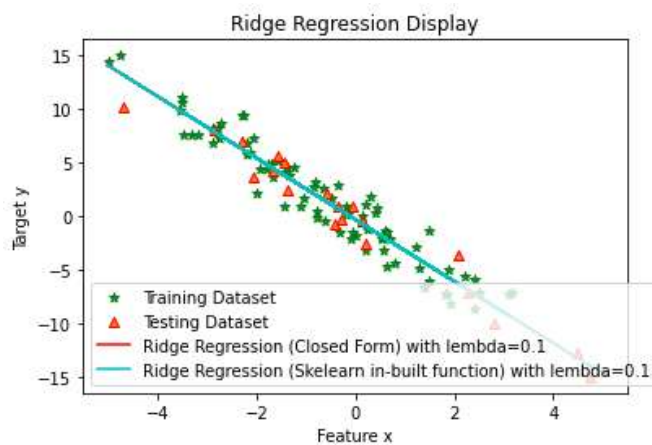
1.537599647698043

RMSE for Testing Ridge Regression with Closed Form Solution =

1.3509952830888237

Ridge Regression Co-efficient $[w_0, w_1]$ using sklearn in-built function

$[-0.32887451429201, \text{array}([-2.87983328])]$



regularization term =

0.5

Ridge Regression Co-efficient $[w_0, w_1]$ using Closed Form Solution =

$[-0.32460118 \ -2.8756468]$

RMSE for Training Ridge Regression with Closed Form Solution =

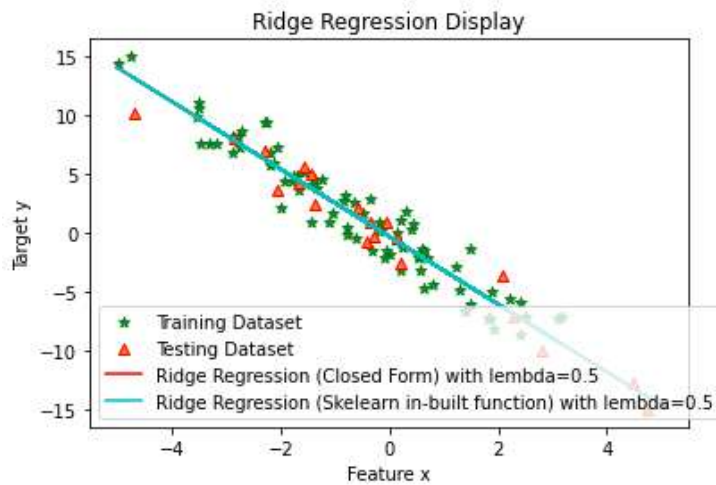
1.537631636236921

RMSE for Testing Ridge Regression with Closed Form Solution =

1.3499409003535163

Ridge Regression Co-efficient [w0, w1] using sklearn in-built function

`[-0.32678758770191374, array([-2.87594084])]`



regularization term =

1.0

Ridge Regression Co-efficient [w0, w1] using Closed Form Solution =

`[-0.31987819 -2.87051152]`

RMSE for Training Ridge Regression with Closed Form Solution =

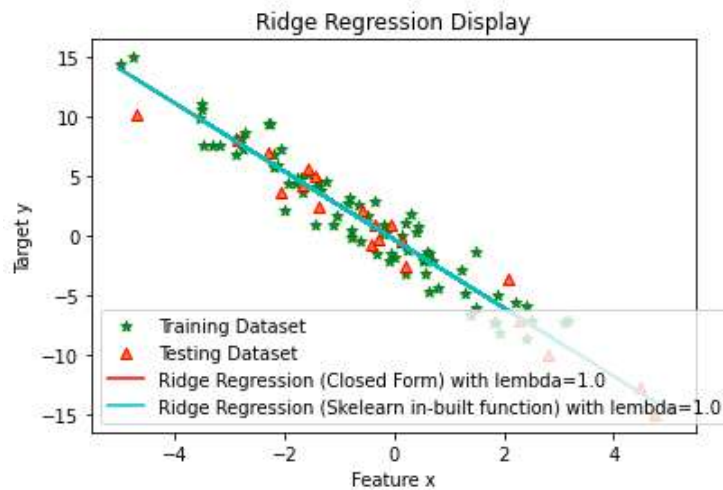
1.5377308191106036

RMSE for Testing Ridge Regression with Closed Form Solution =

1.3487274581991207

Ridge Regression Co-efficient [w0, w1] using sklearn in-built function

`[-0.32418684940243025, array([-2.87109005])]`



regularization term =

1.5

Ridge Regression Co-efficient $[w_0, w_1]$ using Closed Form Solution =

$[-0.31522664 \ -2.86540186]$

RMSE for Training Ridge Regression with Closed Form Solution =

1.5378946741711343

RMSE for Testing Ridge Regression with Closed Form Solution =

1.3476285085225836

Ridge Regression Co-efficient $[w_0, w_1]$ using sklearn in-built function

$[-0.32159486954477523, \text{array}([-2.8662556])]$

